

UNIT 1



PROBLEM SOLVING AND ALGORITHM DESIGNING



Q.1: WHAT IS PROBLEM SOLVING?

PROBLEM SOLVING

Problem solving is a procedure to figure out the solution of complex problems. Problem solving is the main process in computer programming, where programmers first understand the problem, plan the solution and then understand how to translate algorithm into something a computer can do, and finally how to write the specific syntax or code required by a computer to get the job done.

Q.2: DEFINE PROBLEM?

PROBLEM

Problem sometimes referred as an issue, is any situation that occurs and is unexpected or prevents something from occurring. In programming a problem is considered as a matter which is difficult to solve, it is a complex task or routine.

Q.3: WHAT ARE THE STRATEGIES OF PROBLEM SOLVING?

PROBLEM SOLVING PROCESS

Finding a suitable solution for issues can be accomplished by the following the basic four step problem-solving process given below.

- Define the Problem
- Generate alternative solutions
- Select an alternative
- Implement the solution

DEFINE THE PROBLEM

Specifically, the task of defining the problem consists of identifying what you know (input-given data), and what you want to obtain (output-the result). During this step, it is more important to understand described problem.

GENERATE ALTERNATIVE SOLUTIONS

Many alternative solutions to the problem should be generated to achieve the desired results. Generating multiple alternatives can enhance the value of best solution. So, it is good to generate a list of all possible solutions and evaluate to decide which one is the best for that particular problem.

EVALUATE AND SELECT AN ALTERNATIVE

Good problem-solving approach uses a series of considerations when selecting the best alternative. In this step, evaluate all possible solutions related to a target standard and select the best alternative. A particular alternative will solve the problem without causing other unanticipated problems.

IMPLEMENT AND FOLLOW UP ON THE SOLUTION

The implementation of best solution also includes planning on what happens when something goes wrong with the selected solution. It is also important to follow up and track the results of solution at every stage of implementation.

Q.4: WHAT IS ALGORITHM

ALGORITHM

An algorithm is a procedure or formula for solving a problem. It can be defined as the set of welldefined step by step instructions to solve a program. In computer science an

algorithm usually means a small procedure that solves a specific problem designs; algorithms are widely used throughout all areas of IT. There are two common methods to represent algorithm designs; pseudocode and flowcharts.

Q.5: EXPLAIN THE ROLE OF ALGORITHM IN PROBLEM SOLVING. OR DESCRIBE THE ADVANTAGES OF ALGORITHM.**ROLE OF ALGORITHM IN PROBLEM SOLVING**

Algorithms give us the most ideal option of accomplishing a task in problem solving. Algorithms are used to find the best possible way of solving a problem. In doing so they improve the efficiency of a program. With the best algorithm, a computer program will be able to produce very accurate results. An algorithm can be used to improve the speed at which a program executes.

Q.6: WHAT ARE THE QUALITIES OF A GOOD ALGORITHM?**QUALITIES OF A GOOD ALGORITHM**

1. Inputs and outputs should be defined exactly.
2. Every step in an algorithm must be clear, precise and unambiguous.
3. It should be effective among many different ways to solve a problem.
4. It shouldn't have computer code. Instead, the algorithm must be written in such away that, it can be used in similar programming languages.

Q.7: WRITE SOME EXAMPLES OF ALGORITHM.**ALGORITHM EXAMPLES****ALGORITHM1: MAKING A CUP OF TEA**

- Step1:** Start.
Step3: Fill the kettle with water.
Step4: Boil the water in the kettle.
Step5: Pour some of the boiled water into the cup.
Step6: Add milk to the cup.
Step7: Add sugar to the cup.
Step8: Stir the tea.
Step9: Drink the tea.
Step10: Stop

ALGORITHM2: SUBTRACTION OF TWO NUMBERS

- Step1:** Start
Step2: Declare variables n1, n2 & sub
Step3: Read values n1 & n2
Step4: Subtract n1 & n2 and assign the result to sub
sub=n1-n2
Step5: Display sub
Step6: Stop

ALGORITHM3: ADDITION OF TWO NUMBERS

- Step1:** Startmjeecoaching.blogspot.com
Step2: Declare variables n1, n2 & sum
Step3: Read values n1 & n2

Step 4: Add n1 & n2 and assign the result to

sum = n1+n2

Step 5: Display sum

Step 6: Stop

ALGORITHM 4: MULTIPLICATION OF TWO NUMBERS

Step 1: Start

Step 2: Declare variables n1, n2 & multiply

Step 3: Multiply n1 & n2 and assign the result to multiply.

multiply = n1*n2

Step 4: Display multiplication

Step 5: Stop

ALGORITHM 5: AREA OF CIRCLE

Step 1: Start

Step 2: Declare variable r and area

Step 3: Read value of r

Step 4: Apply formula {Area = (3.142) * r*r}

area (3.142) * r*r

Step 5: Display area

Step 6: Stop

Q.8: WHAT IS FLOWCHART?

FLOWCHART

A flowchart is a pictorial representation of an algorithm, workflow or process, showing the steps as boxes of various kinds, and their order by connecting them with arrows. It is used to show the sequence of steps and logic of solution to a given problem. Flowcharts are used in analyzing, designing, documenting or managing a process or program in various fields.

Q.9: DEFINE DIFFERENT SYMBOLS USED

FLOWCHART SYMBOLS

1. TERMINAL OR START/STOP

The terminal symbol is used to indicate the beginning (START) and ending (STOP) in the Process symbol is used to illustrate a process, action or an operation. These are represented by rectangles.

2. INPUT/OUTPUT

The input/output symbol is used to denote any type of input data or output information.

3. DECISION

The decision symbol is used in a flowchart to indicate a point at which a decision has to be made and a branch to one of two or more alternative points are possible.

4. ARROWS OR FLOWLINES

Flowlines with arrowheads are used to indicate the flow of operation. It shows the exact sequence in which the instruct are to be executed.

Q.10: WRITE DOWN THE IMPORTANCE OF FLOWCHART IN PROBLEM SOLVING.

IMPORTANCE OF FLOWCHART IN SOLVING A PROBLEM

Flowcharts are better way of communicating the logic of a system.

EFFECTIVE ANALYSIS

With the help of flowchart, problem can be analyzed in more effective way therefore reducing cost and wastage of time.

EFFICIENT CODING

The flowcharts act as a guide or blueprint during the systems analysis and program development phase.

PROPER DEBUGGING

The flowchart helps in debugging a process.

EFFICIENT PROGRAM MAINTENANCE

The maintenance of operating program becomes easy with the help of flowchart. It helps the programmer to put efforts more efficiently on that part.

Q.11: WRITE DOWN THE EXAMPLE OF FLOWCHART.

FLOWCHART EXAMPLE

Step 1: Start

Step 2: Input Number 1

Step 3: Number 2

Step 4: Sum = Number 1 + number 2

Step 5: Print Sum

Step 6: Stop

Q.12: WHAT ARE THE DIFFERENCES BETWEEN ALGORITHM AND FLOWCHART?

DIFFERENCES BETWEEN ALGORITHM & FLOWCHART

Feature	Algorithm	Flowchart
Definition	A set of step-by-step instructions for solving a problem.	A visual representation of an algorithm using shapes and arrows.
Level of Detail	Can be more detailed, including specific data structures and logic.	Provides a high-level overview of the process.
Accuracy	Can be more precise and unambiguous.	Can be open to interpretation depending on complexity.
Error Debugging	Difficult to identify errors in complex algorithms.	Easier to visualize and identify potential errors in the process flow.
University	Can be expressed in various ways (pseudocode, natural language)	Uses standardized symbols and connectors.

Q.13: DEFINE DATA STRUCTURES.

DATA STRUCTURES

Data structure is a particular means of organizing and storing data in computers in such a way that we can perform operations on the stored data more efficiently. Data structures have a wide and diverse scope of usage across the fields of Computer Science and Software engineering. Data structure may be linear or non-linear.

Q.14: DESCRIBE LINEAR DATA STRUCTURE.

LINEAR DATA STRUCTURE

Linear data structure arranges the data in a linear fashion. The data elements are arranged sequentially such that the element is directly linked to its previous and the next elements. This data structure supports single-level storage of data. And hence, transfer of the data is achieved through a single run only.

Examples of linear data structures are Stack, Queue, Array, etc.

Q.15: DEFINE STACK AND ITS OPERATIONS.

Stack is a linear data structure that stores information in such a way that the last item stored is the first item retrieved. It is based on the principle of LIFO (Last-in-first-out). The stack in digital computers is a group of memory locations with a register that holds the address of top of element.

Stacks usually performs two operations.

Push: It is used to insert an element on top of stack.

Pop: It is used to delete an element from top of stack.

Q.16: DEFINE QUEUE DATA STRUCTURE AND ITS OPERATIONS.

QUEUE

Queue is the type of linear data structure where the elements to be stored follow the rule of First in First Out (FIFO). In queue, the element that was added first is removed first) In this data structure, both ends are used for the insertion and the removal of data. The two main operations of queue are enqueue and dequeue.

Enqueue: It refers to the process where inserting an element is allowed to the collection of data.

Dequeue: It refers to the process where removal of elements is allowed, which element of queue.

Q.17: DEFINE ARRAY DATA STRUCTURE AND ITS OPERATIONS.

ARRAY

The array is the type of linear data structure that stores homogeneous (same type) elements at memory locations which are consecutive. The same types of objects are stored sequentially in an array. The main idea of an array is that multiple data of the same type can be stored together. Before storing the data in an array, the size of the array has to be defined. The location of elements stored in an array has a numerical value called index to identify the element. Some operations of array are:

Traverse: Go through the elements and print them.

Search: Search for an element in the array by its value or its index.

Update: Update the value of an existing element at a given index.

Insert: Insert an element in array.

Deletion: Delete an element from array.

Sorting: Rearrange a given array or list of elements.

Memory Locations												
200	201	202	203	204	205	206	207	208	209	210	211	
11	9	17	89	1	90	19	5	3	23	43	99	
0	1	2	3	4	5	6	7	8	9	10	11	
Index												